

文献引用格式: 王宇, 刘浪华. 基于 SWD 协议的 ISP 技术研究与应用 [J]. 通信技术, 2021, 54 (7): 1800–1806.

WANG Yu, LIU Langhua. Research and implementation of ISP technology based on SWD [J]. Communications Technology, 2021, 54(7): 1800–1806.

doi:10.3969/j.issn.1002-0802.2021.07.039

基于 SWD 协议的 ISP 技术研究与应用^{*}

王 宇, 刘浪华

(中国电子科技集团公司第三十研究所, 四川 成都 610041)

摘 要: 传统嵌入式应用中, 不便于对微处理器芯片固件进行升级, 影响产品后期的维护。通过对 ARM 嵌入式核调试功能与串行调试 (Serial Wire Debug, SWD) 协议的研究, 实现了芯片 ARM 内核控制与访问。然后经过对芯片 Flash 算法文件 FLM 的研究解析, 将提取的 Flash 相关二进制可执行码在 ARM 核内部静态随机存储寄存器 (Static Random Access Memory, SRAM) 中恢复, 利用 ARM 核心寄存器实现参数与 Flash 函数调用, 完成主设备对芯片固件的在线系统编程 (In-System Programming, ISP) 烧录更新, 并通过编程验证了 SWD 协议时序并实现了应用。

关键词: SWD; ARM; ISP 编程; 嵌入式系统

中图分类号: TP311.5 **文献标识码:** A **文章编号:** 1002-0802(2021)-07-1800-07

Research and Implementation of ISP Technology based on SWD

WANG Yu, LIU Langhua

(No. 30 Institute of CETC, Chengdu Sichuan 610041, China)

Abstract: In traditional embedded applications, it is inconvenient to upgrade the microprocessor chip firmware, which affects the later maintenance of the product. With the study of ARM embedded core debugging function and SWD (Serial Wire Debug) protocol, the control and access of ARM core registers are realized. Then, through the research and analysis of the chip Flash algorithm file FLM, this article restores the extracted Flash-related binary executable code in the SRAM (Static Random Access Memory) inside the ARM core. It uses ARM core registers to implement parameters and Flash function calls, completes the in-system programming (ISP) burning and updating of the chip firmware by the host device, and verifies the SWD protocol timing through programming and realizes the application.

Keywords: SWD; ARM; ISP (In System Programming); embedded system

0 引 言

目前嵌入式领域的主流产品在研发阶段主要通过仿真器对微处理器进行程序的仿真验证, 生产阶段通过专用联合测试工作组 (Joint Test Action Group, JTAG) 烧录器完成芯片固件烧录, 基本不

涉及后期固件的升级维护。一旦固件程序在应用上存在漏洞, 只能将设备返厂维修, 导致产品后期维护费用居高不下。通过结合串行调试 (Serial Wire Debug, SWD) 串行调试技术和在线系统编程 (In-System Programming, ISP) 技术, 能够解决设备后

* 收稿日期: 2021-03-10; 修回日期: 2021-06-09 Received date: 2021-03-10; Revised date: 2021-06-09

期维护不便的问题。

SWD 技术支持所有可通过 JTAG 接口进行调试的 ARM Cortex A/R/M 系列处理器和具有 CoreSight 调试架构的 ARM 芯片^[1], 包括德州仪器 (Texas Instrument, TI)、恩智浦 (NXP Semiconductors)、Atmel、Samsung 等厂家的芯片, 以及意法半导体 (STMicroelectronics, STM) 公司 STM32 系列微处理器等。

1 SWD 协议原理研究

SWD 串行调试由一个双向数据线和单向时钟

线组成实现数据的同步传输, 可替代 JTAG 调试。时钟频率为 50 MHz 时数据传输速率可达 4 MB/s, 较少的印制电路板 (Printed Circuit Board, PCB) 布局布线资源便可实现低功耗低成本应用^[2]。SWD 协议实现芯片资源的交互流程如图 1 所示。

1.1 SWD 接口协议

SWD 操作流程主要分为 3 个阶段: 数据请求包 REQ、应答数据包 ACK 与数据读写传输包^[3]。表 1 列举了 SWD 接口交互流程的主要关键字信息。

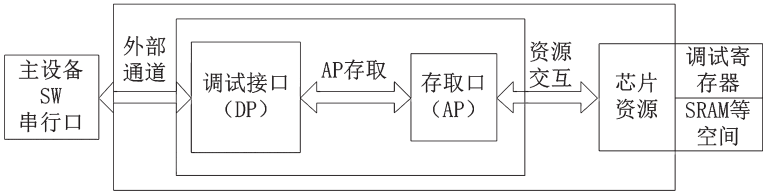


图 1 SWD 协议数据交互流程

表 1 SWD 协议关键词

关键字	说明
Start	起始位, 默认为 1
AP/DP	单比特位, 0 表示 DP 访问模式, 1 表示 AP 访问模式
RnW	单比特位, 0 表示写操作, 1 表示读操作
A[2:3]	双比特位, 最低有效位在前, 选择 DP/AP 寄存器访问地址
Parity	单比特奇偶校验位
Stop	单比特停止位, 恒为 0
Park	单比特位, 主设备不对数据线进行驱动, 由 SWD 硬件接口拉高
Turn	单比特位, 主设备放弃对 SWD 总线的控制, 默认为一个时钟周期
ACK[0:2]	3 bit 从设备响应位, 最低有效位在前。值为 b100 时, 操作正常, 其他则异常
WDATA[0:31]	32 位写数据, 主设备到从设备, 以最低有效位在前的方式发送
RDATA[0:31]	32 位读数据, 从设备到主设备, 以最低有效位在前的方式接收

SWD 的写操作由 1 byte 请求包、3 bit 应答包与 33 bit 的写数据组成, 一次完整的写操作如图 2 所示。
SWD 的读操作由 1 byte 的请求包、3 bit 的应答

包与 33 bit 的读数据组成, 应答包与读数据包间无 Turn 周期, 一次完整的读操作如图 3 所示。

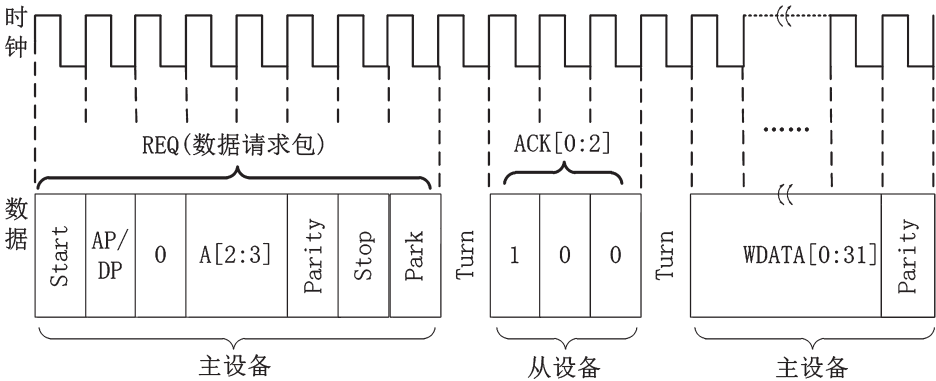


图 2 SWD 协议写操作时序

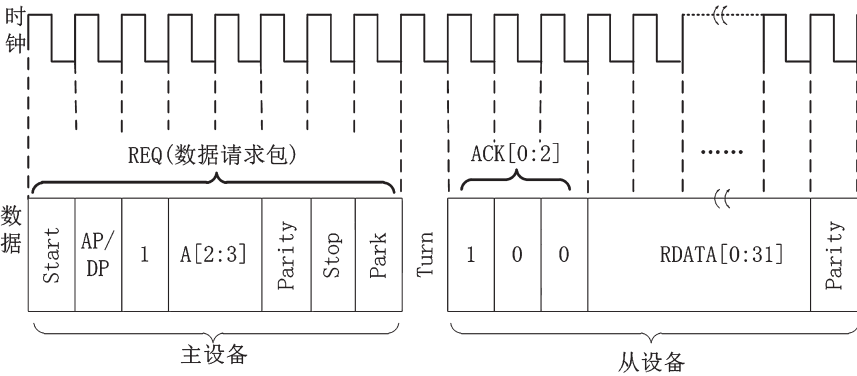


图3 SWD 协议读操作时序

SWD 中 DP 寄存器组成见表 2，其中 ID 标识寄存器用于验证 SWD 操作时序，获取 ARM 调试接口信息等，低 12 位固定为 0×477，主要构成如图 4 所示。

表 2 SW-DP 寄存器

A[3:2]	CTRLSEL	读写	功能描述
00	X	读	ID 标识寄存器 IDCODE
		写	AP 终止寄存器 ABORT
01	0	读 / 写	控制 / 状态寄存器 CTRL/STAT
	1	读 / 写	线控制寄存器 WCR
10	X	读	读重传寄存器 RESEND
		写	AP 选择寄存器 SELECT
11	X	读	读缓冲寄存器 RBUF

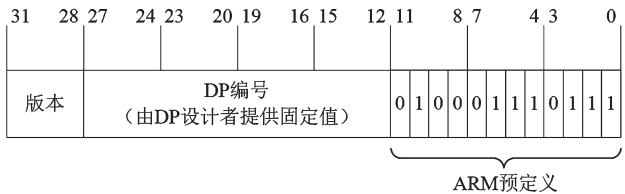


图4 IDCODE 寄存器

SELECT 寄存器控制 AP 选择特定的地址，其最低位为 CTRLSEL，其组成见图 5。[31:24] 位

APSEL 值确定当前 AP 地址的高 8 bit；每个 AP 具有 16 个 Bank，[7:4] 位 APBANKSEL 值（0-F）用来选择当前 AP 的 Bank 号；每个 Bank 有 4 个 32 bit 寄存器，由数据请求包 REQ 中的 A[3:2] 位来确定。



图5 SELECT 寄存器

图 6 列举了每组 MEM-AP 寄存器的信息，其中 Bank 0x0 中的 CSW 寄存器、TAR 寄存器与 DRW 寄存器对设备 ARM 核的数据传递起着关键作用^[4]。

1.2 SWD 实现处理器资源传递

SWD 的数据传递通过分级传递的方式：第一级通过 DP 接口选择进入 AP 寄存器，然后通过 AP 接口选择目的寄存器进行访问；第二级配置 CSW 寄存器进行软件控制与地址模式的选择，最后通过 TAR 寄存器与 DRW 寄存器对处理器目标地址进行读写。整个交互流程如图 7 所示。

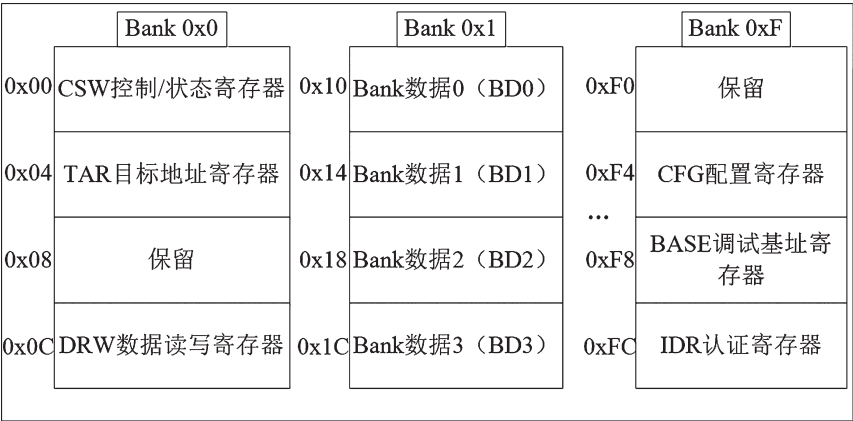


图6 MEM-AP 寄存器

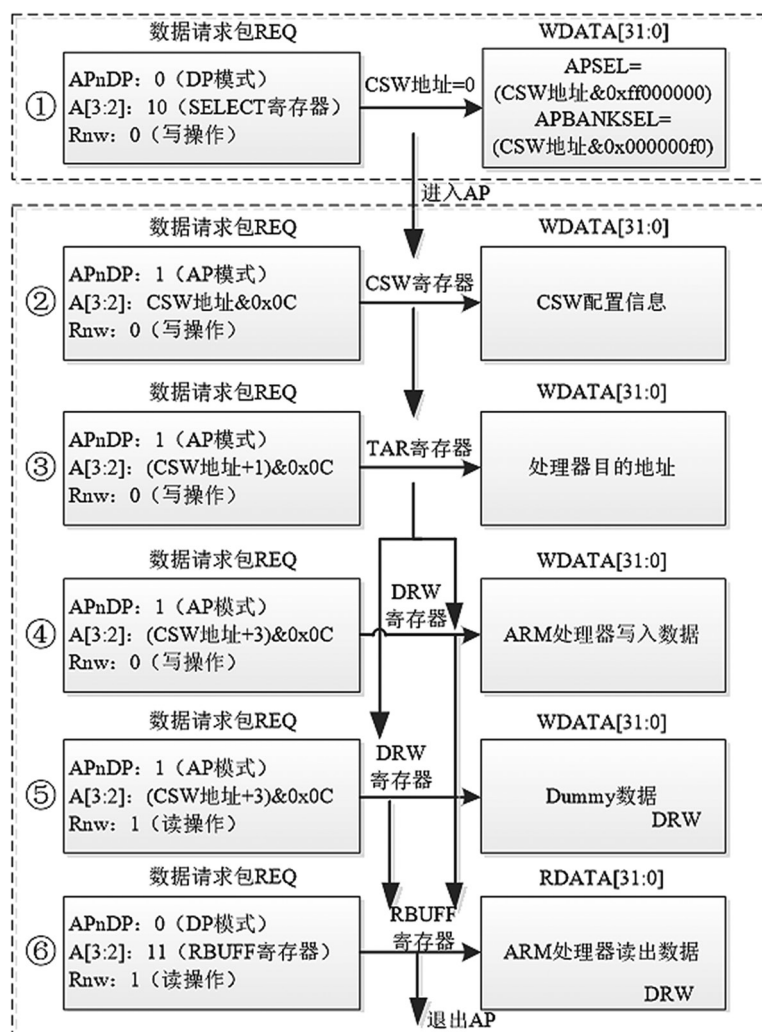


图 7 SWD 读写处理器资源实现

以下是实现的主要函数。

(1) Func0: 单 32 bit 数据写入, 经历①②③④⑥这 5 个阶段, 参数为其③阶段的目的地址与④阶段的数据, ⑥阶段仅需执行读操作。

(2) Func1: 连续 32 bit 数据写入, SWD 协议中支持连续写入, 目的地址会在写入完成后自加 1, 大致流程与 Func0 一致, 经历①②③④...④(n 个④)⑥阶段, 其中④阶段的个数 n 由写入数据数量来确定。

(3) Func2: 单 32 bit 数据读取, 经历①②③⑤⑥这 5 个阶段, 参数为其③阶段的目的地址与接收⑥阶段的数据缓存。

(4) Func3: 连续 32 bit 数据读取, SWD 协议中目的地址会在读取完成后自加 1, 大致流程 Func2 一致, 经历①②③⑤⑥...⑥(n 个⑥)阶段,

其中⑥阶段的个数 n 由读取数据数量来确定。

2 STM32F103 核心资源

为了利用 SWD 接口完成对 ARM-Cortex M3/M4 等芯片固件的擦除与烧写, 实现固件的更新。本节以 ST 公司的 STM32F103x 系列芯片作为说明, 对芯片内部资源架构进行基本介绍^[5]。

如图 8 所示, STM32F103x 采用 ARM Cortex-M3 核, 采用 32 位 RISC 精简指令集, 时钟频率最高 72 MHz。Flash 容量最大 512 kB、SRAM 最大 64 kB, STM32F103x 芯片内部存储资源如图 8 所示, 其中 Flash 区存储了芯片的 BootLoader 与主程序。DAP 烧录器、J-link 编程器进行芯片固件烧录也是在该区域完成。

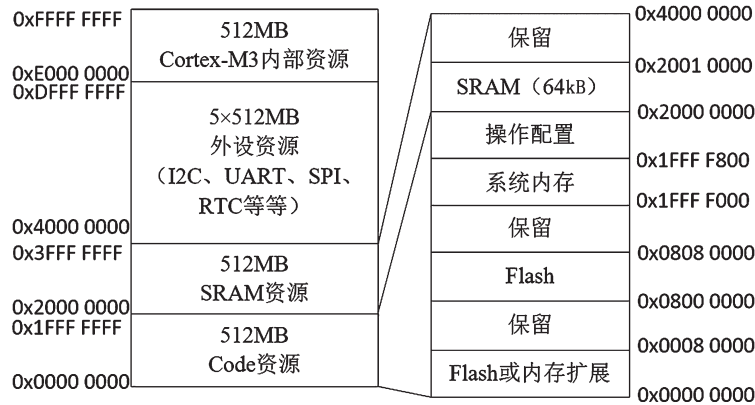


图 8 STM32F103x 内部资源映射

通过 SWD 接口可实现 STM32F103x 芯片 SRAM 区域存取访问，控制 Cortex-M3 内部资源中调试寄存器，读取 Flash 区的数据。Flash 区不能直接进行数据擦写，必须通过特定方法，本文研究的主要目的就是通过 SWD 与芯片 Flash 算法文件 FLM 相结合的方式实现芯片固件更新。

Cortex-M3/M4 的 ARM 核调试寄存器地址完全相同，寄存器相关描述如表 3 所示。DFSR 寄存器用来获取 ARM 核运行错误状态，DHCSR 与 DEMCR 可以控制 ARM 核的运转状态，操作 SRAM 区域或者 Flash 资源时都应将 ARM 核挂起，以免影响数据正确性。DCRSR 寄存器用来选择 ARM 核心寄存器，DCRDR 寄存器用来管理 ARM 核心寄存器值^[6]。

表 3 Cortex-M3/M4 调试寄存器说明

地址	名称	功能描述
0xE000ED30	DFSR	错误状态寄存器
0xE000EDF0	DHCSR	挂起控制 / 状态寄存器
0xE000EDF4	DCRSR	ARM 核寄存器选择寄存器
0xE000EDF8	DCRDR	ARM 核寄存器数据寄存器
0xE000EDFC	DEMCR	异常监控控制寄存器

ARM 核心寄存器组成如表 4 所示，其中 R0 ~ R12 为 32 位通用目的寄存器，R13 可对堆栈空间进行存取操作，R14 链接寄存器用于存储子程序或者函数调用的返回地址，R15 程序计数器读操作返回当前正在执行的指令加上 4，写入 R15 会导致程序跳转执行。XPSR 程序状态寄存器作为特殊功能寄存器用来判断 ARM 核心寄存器执行状态是否正常。

表 4 ARM 核心寄存器

寄存器	功能描述
R0 ~ R12	通用寄存器
R13	SP 栈指针
R14	LR 链接寄存器
R15	PC 程序计数器
XPSR	状态寄存器

汇编可执行程序就是在 ARM 核心寄存器上进行变量的计算与处理。通过调用 Func0、Func2 流程可以控制 Cortex-M3/M4 的所有调试寄存器，经由 DCRSR 与 DCRDR 寄存器实现 ARM 核心寄存器读取，获取到 ARM 核的执行控制权限。

3 SWD 实现芯片固件 ISP 在线编程

通用 Flash 芯片编程流程包括：初始化、擦除与烧写。SWD 协议调用 FLM 文件中可执行二进制文件的方式实现 Flash 的 ISP 编程。FLM 文件从芯片厂家获取，由 Flash 擦写相关函数及变量等组成的通用标准文件格式可执行文件编码。本文以 STM32F103 芯片的 STM32F10x_128.FLM 文件为样本进行研究与应用，可推广至所有支持 SWD 协议芯片进行应用。

3.1 STM32F10x_128 算法文件解析

STM32F10x_128.FLM 文件可通过 elfparser 等软件解析，也可在 Linux Ubuntu 操作系统下输入以下命令查看解析文件信息 readelf -a STM32F10x_128.FLM。

截取的部分重要信息如图 9 所示，其中 Prgcode 程序代码段包括 Flash 的若干函数执行码，PrgData 程序数据段为全局变量 / 静态变量区间。SWD 所需的核心内容（Prgcode+PrgData）数据段可由地址 Addr、偏移 Offset 与大小 Size 从 FLM 件中获得。

3.2 STM32F10x_128 读取 IDCODE

图 10 由示波器获得，下方波形为 SWD 时钟，上方波形为 SWD 数据。以数据线的第一个高电平为 Start 起始位，对比 SWD 读操作原理图，最终获取的内容为 0x2ba01477。参照 1.1 节 DP 寄存器中有关 IDCODE 的描述，能够验证获取到内容确实为 IDCODE 的值，进而确认已成功实现对 SWD 接口控制。

```
[root@phytium scripts]# readelf -a STM32F10x_128.FLM
Section Headers:
[Nr] Name              Type              Addr              Off              Size
[ 0] NULL                NULL              00000000          000000          000000
[ 1] PrgCode              PROGBITS          00000000          000034          000128
[ 2] PrgData              PROGBITS          00000128          00015c          000004
[ 3] DevDscr              PROGBITS          0000012c          000160          0010a0
Symbol table '.symtab' contains 16 entries:
Num:  Value            Size  Type  Bind  Vis  Ndx  Name
10:  00000001            50  FUNC  GLOBAL  HIDDEN  1  Init
11:  00000033            18  FUNC  GLOBAL  HIDDEN  1  UnInit
12:  00000045            58  FUNC  GLOBAL  HIDDEN  1  EraseChip
13:  0000007f            62  FUNC  GLOBAL  HIDDEN  1  EraseSector
14:  000000bd            90  FUNC  GLOBAL  HIDDEN  1  ProgramPage
```

图 9 FLM 文件解析内容

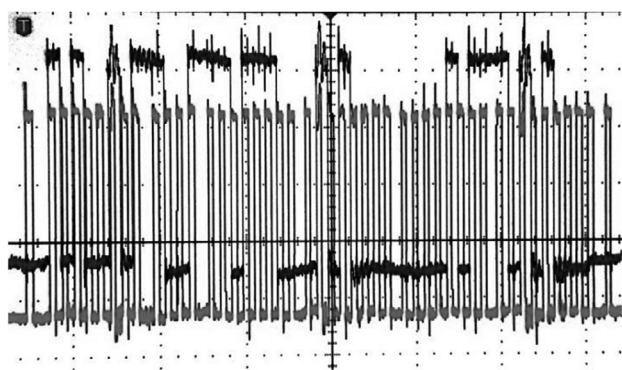


图 10 SWD 读取 IDCODE 波形图

3.3 芯片 Flash 固件烧写

通过 SWD 接口实现 ARM 核 Flash 资源烧写共分为 3 个步骤:

(1) 通过调试寄存器挂起 ARM 核, 停止 ARM 核的运行, 将 FLM 算法文件提出的可执行数据写入 SRAM 的起始地址;

(2) 将芯片固件 STM32F10x_128.Bin 写入 SRAM 未用空间, 待 Flash 编程时使用;

(3) 将执行函数包括程序入口参数等信息通过调试寄存器写入 ARM 核心寄存器, 依次调用 Flash 初始化、片擦除、编程函数, 完成 ARM 核内部 Flash 的烧录。整个流程执行如图 11 所示。

Flash 固件烧写测试程序由 C 语言实现, 涵盖了 Flash 初始化、片擦除、程序烧写、数据回读校验及退出操作, 顶层实现代码如下文所述:

```
{uint32_t addr;
uint8_t tmpbuf[128*1024];
swd_flash_init(0); //Init
for(addr = 0; addr < lenth; addr += 0x400){
swd_flash_erase_sector(addr); //EraseSector
}
//ProgramPage
swd_flash_program_page(0, buf, lenth);
swd_read_memory(0, tmpbuf, lenth); //Func3
```

```
if (memcmp(tmpbuf, buf, lenth))
{
printf( "Flash burn verify failed.\n" );
return -1;
}
swd_flash_uninit(); //UnInit
printf( "Flash burn verify OK.\n" );
return 0; }
```

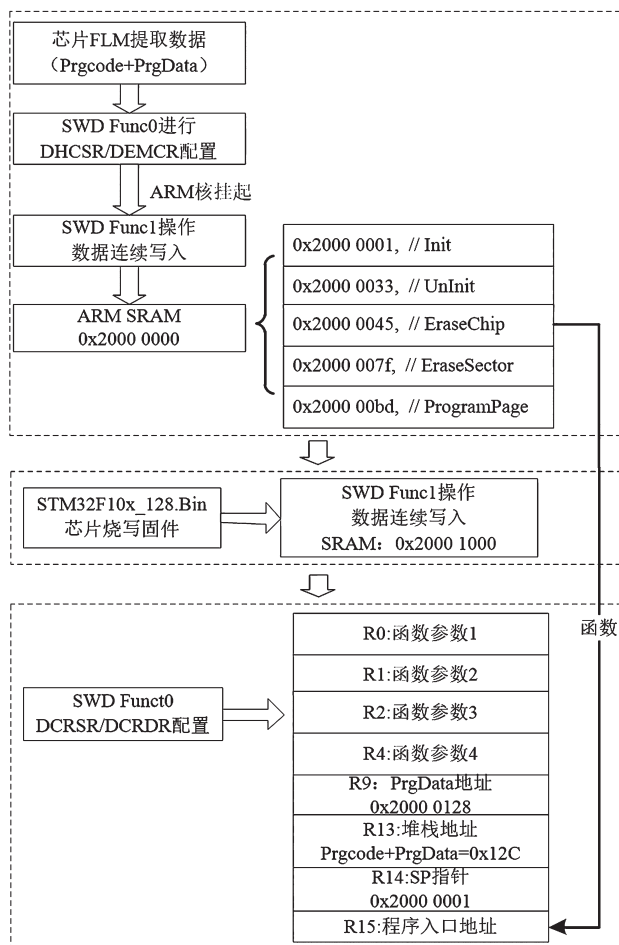


图 11 SWD 接口实现 FLASH 擦写流程

在主设备上执行 Flash 固件烧写测试程序 swdprogram, 稳定地实现了芯片固件 ISP 更新, 程序运行结果如图 12 所示。

```
# ./swdprogram STM32F10x_128.Bin
file_size:21956
[ 1152.742062] Flash burn verify OK.
[ 1152.788154] swd_burn OK.
#
```

图 12 Flash 烧录测试结果

4 结 语

通过 SWD 接口协议控制芯片 ARM 核的运行,结合芯片 FLM 文件解析并在 ARM 核中实现 Flash 相关函数资源的恢复与调用,完成了芯片固件的 ISP 在线编程。针对其它支持 SWD 接口协议的 ARM 芯片,除内部资源地址空间与 FLM 算法文件不同外,ARM 核的调试与控制操作完全相同,很容易推广并应用于主从模式下的微处理器芯片。通过 SWD 接口实现的 ISP 在线编程,能够极大地提高设备的维护性,对于实现设备程序升级与功能更新有较高的应用价值。

参考文献:

- [1] ARM Inc.ARM coresight DAP-lite(technical reference manual)[S].Cambridge:ARM Inc,2013.
- [2] ARM Inc.ARM debug interface V5(architecture specification)[S].Cambridge:ARM Inc,2006.

- [3] 黄亚萍.SWD 协议的研究及 ARM 程序下载器的设计[D].武汉:湖北大学,2012:24-33.
HUANG Yaping.The research of the SWD agreement and the design of program downloader based on ARM[D].Wuhan:Hubei University,2012:24-33.
- [4] 史兴强,鲍宜鹏.基于 HDL 任务的 SWD 协议实现与仿真[J].电子与封装,2019,19(1):32-36.
SHI Xingqiang,BAO Yipeng.Implementation and simulation of the SWD agreement based on HDL task[J].Electronics and Packaging, 2019,19(1):32-36
- [5] STMicroelectronics Inc.STM32F103xC, STM32F103xD, STM32F103xE data sheet(production data)[S].Geneva:STMicroelectronics Inc,2018.
- [6] ARM Inc.Cortex-M3 technical reference manual[S].Cambridge:ARM Inc,2010.

作者简介:



王 宇(1987—),男,硕士,工程师,
主要研究方向为嵌入式开发;

刘浪华(1986—),男,硕士,工程师,
主要研究方向为嵌入式开发。

声 明

《通信技术》编辑部从未授权任何单位及个人代为收取任何费用,有任何经济往来只能通过对公账户,请广大作者和读者知悉。

唯一官方投稿网站: www.txjszz.com

唯一官方投稿邮箱: tongxinjishu001@163.com

如有其他形式,均为假冒,本社将保留诉讼法律的权利。

特此声明。

《通信技术》编辑部

2021 年 7 月 5 日